

Travlr API Testing Guide

Overview

This guide provides comprehensive instructions for testing the Travlr API using Postman. The API supports full CRUD operations with JWT authentication for administrative functions.

Setup Instructions

1. Import Postman Collection

1. Open Postman
2. Click "Import" button
3. Select the `Travlr_API_Complete_Testing.postman_collection.json` file
4. Collection will be imported with all test cases

2. Environment Setup

The collection includes these variables: - `base_url`: `http://localhost:3000/api` - `auth_token`: (automatically set after login) - `trip_id`: (automatically set from trip responses)

3. Prerequisites

- Backend server running on port 3000
- MongoDB connected and seeded with sample data
- Admin user created (admin@travlr.com / admin123)

Test Categories

Authentication Tests

Test	Method	Endpoint	Purpose
Login Admin	POST	/auth/login	Get JWT token for admin operations
Register New User	POST	/auth/register	Create new user account

Public Trip Tests

Test	Method	Endpoint	Purpose
Get All Trips	GET	/trips	Retrieve all trips (no auth required)
Get Single Trip	GET	/trips/:id	Retrieve specific trip by ID

Protected Trip Tests (Admin Only)

Test	Method	Endpoint	Purpose
Create New Trip	POST	/trips	Add new trip to database
Update Trip (PUT)	PUT	/trips/:id	Complete trip update
Partial Update (PATCH)	PATCH	/trips/:id	Update specific fields only
Delete Trip	DELETE	/trips/:id	Remove trip from database

Error Handling Tests

Test	Expected Status	Purpose
Unauthorized Access	401	Verify protected endpoints require auth
Invalid Login	401	Test wrong credentials handling
Trip Not Found	404	Test non-existent resource handling

Data Validation Tests

Test	Expected Status	Purpose
Missing Required Fields	400	Test input validation
Duplicate Trip Code	400/409	Test unique constraint validation



Running Tests

Step 1: Authentication

1. Run "Login Admin" test first
2. Verify JWT token is automatically stored
3. All subsequent protected tests will use this token

Step 2: Public Operations

1. Run "Get All Trips" to verify basic connectivity
2. Run "Get Single Trip" to test individual trip retrieval

Step 3: CRUD Operations

1. **CREATE:** Run "Create New Trip" test
2. **READ:** Verify trip appears in "Get All Trips"
3. **UPDATE:** Run "Update Trip (PUT)" test
4. **PATCH:** Run "Partial Update (PATCH)" test
5. **DELETE:** Run "Delete Trip" test

Step 4: Error Handling

1. Run all error handling tests
2. Verify appropriate status codes and error messages

Step 5: Data Validation

1. Test missing required fields
2. Test duplicate code validation



Expected Results

Successful Operations

- **Login:** Status 200, JWT token returned
- **GET Trips:** Status 200, array of trips
- **POST Trip:** Status 201, created trip with ID
- **PUT Trip:** Status 200, updated trip data
- **PATCH Trip:** Status 200, partially updated trip
- **DELETE Trip:** Status 200, deletion confirmation

Error Responses

- **401 Unauthorized:** Missing or invalid token
- **400 Bad Request:** Invalid input data
- **404 Not Found:** Resource doesn't exist
- **409 Conflict:** Duplicate trip code



Test Automation

Each test includes automated assertions: - Status code verification - Response structure validation - Data integrity checks - Token management - Variable extraction for chained tests

Security Testing

The collection tests: - JWT token authentication - Protected endpoint access control - Token expiration handling - Unauthorized access prevention

Sample Test Data

Admin Login

```
{  
  "email": "admin@travlr.com",  
  "password": "admin123"  
}
```

Sample Trip Creation

```
{  
  "code": "TEST001",  
  "name": "Test Adventure",  
  "length": "3 days / 2 nights",  
  "start": "2024-07-01",  
  "resort": "Test Resort, 4 stars",  
  "perPerson": "$999.00",  
  "image": "https://via.placeholder.com/400x250?text=Test+Trip",  
  "description": "Test trip for API validation"  
}
```

Troubleshooting

Common Issues

1. **Server Not Running:** Ensure backend is running on port 3000
2. **Database Connection:** Verify MongoDB is running and seeded
3. **Token Expired:** Re-run login test to get fresh token
4. **CORS Errors:** Verify CORS is configured for Postman

Debug Tips

- Check Postman console for detailed error logs
- Verify environment variables are set correctly
- Ensure proper Content-Type headers
- Check server logs for backend errors



Performance Testing

The collection can be used with Postman's Collection Runner for: - Load testing with multiple iterations - Performance monitoring - Automated regression testing - CI/CD integration



Continuous Testing

For ongoing development: 1. Run full collection after code changes 2. Use Newman CLI for automated testing 3. Integrate with CI/CD pipelines 4. Monitor API performance metrics



Support

For issues with the API testing: 1. Check server logs for backend errors 2. Verify database connectivity 3. Ensure all dependencies are installed 4. Review Postman console for request details